

BIDRA PLATFORM

Master AI Context

Version: 1.0

Purpose

This document is the primary context loaded before every AI development session.

It summarizes the platform architecture, business rules and engineering principles.

Detailed implementation requirements remain in the specification volumes.

1. Platform Overview

Bidra is a community contribution platform where organizations and individuals create campaigns and needs, and people, companies or other organization contribute through multiple contribution types.

Supported contribution types:

- Financial
- Volunteer
- Skills
- Goods
- Equipment
- Sponsorship
- Venue
- Transport
- Awareness
- Custom

Financial contributions are processed through Stripe Connect.

Volunteer work earns Time Credits.

Time Credits are redeemable with Recognition Partners.

2. Core Domain Hierarchy

The domain hierarchy must never change.

Platform

└─ Users

└─ Organizations

└─ Campaigns

└─ Needs

└─ Contributions

Everything revolves around Needs.

Contributions are always linked to a Need.

Campaigns never receive contributions directly.

3. Core Principles

Always build using:

- Domain-driven design
- Modular architecture
- API-first development
- Event-driven communication
- Strong typing
- Secure-by-default principles

Every module owns its own business logic.

Never duplicate business rules.

4. Platform Modules

The platform consists of these major modules:

1. Authentication
2. Organizations
3. Campaigns
4. Needs
5. Contributions
6. Payments
7. Volunteers
8. Time Credits
9. Recognition

10. Administration

11. Analytics

Modules communicate through APIs and domain events.

Avoid tight coupling.

5. Technology Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- TanStack Query

Backend

- NestJS
- Prisma
- PostgreSQL
- Redis
- BullMQ

Infrastructure

- Docker
 - GitHub Actions
 - OpenAPI
 - Meilisearch
 - Stripe Connect
-

6. Architectural Rules

Always respect module boundaries.

Authentication owns:

- Users
- Roles
- Sessions
- Permissions

Organizations own:

- Organization profiles
- Membership
- Verification

Campaigns own:

- Campaigns
- Needs

Contributions own:

- Contribution workflows

Payments own:

- Stripe
- Financial Ledger

Volunteer owns:

- Attendance
- Time Credits

Recognition owns:

- Rewards
- Vouchers

Admin owns platform operations only.

Never place business logic in another module.

7. Database Rules

Use PostgreSQL with Prisma.

Every table:

- UUID primary key
- Audit timestamps
- Soft delete where appropriate
- Foreign keys
- Constraints
- Indexes

Never bypass Prisma unless absolutely necessary.

All schema changes require migrations.

8. Public IDs

Never expose database UUIDs.

Public resources use immutable Public IDs.

Examples:

Organization

ORG-xxxxxx

Campaign

CAM-xxxxxx

Need

NED-xxxxxx

Contribution

CON-xxxxxx

QR codes and public URLs always use Public IDs.

9. API Rules

REST API only.

Every endpoint must include:

- Validation
- Authorization
- Typed DTOs
- OpenAPI documentation
- Standard responses
- Standard error handling

Never trust frontend validation.

10. Authentication Rules

Authentication uses:

- JWT Access Tokens
- Refresh Tokens
- Session Management
- Role-Based Access Control
- Permission-Based Authorization

All authorization is enforced on the backend.

11. Financial Rules

Financial correctness is critical.

Rules:

- Ledger is immutable.
 - Never edit ledger entries.
 - Never delete ledger entries.
 - Corrections create reversing entries.
 - Stripe webhooks are the source of truth for payment completion.
 - Frontend redirects never confirm payments.
-

12. Time Credit Rules

Time Credits are not money.

Rules:

- Ledger is immutable.
 - Wallet balance is derived from ledger entries.
 - Credits cannot be bought.
 - Credits cannot be transferred.
 - Credits cannot be exchanged for cash.
-

13. Security Rules

Always:

- Validate input
- Validate permissions
- Audit sensitive actions
- Hash passwords with Argon2id
- Verify Stripe signatures
- Rate-limit sensitive endpoints

Never:

- Log secrets
 - Expose UUIDs
 - Trust client-side validation
-

14. UI Rules

Every page must include:

- Loading state
- Empty state
- Error state
- Responsive design
- Accessibility
- Permission-aware UI

Reuse shared UI components.

Do not duplicate UI patterns.

15. Event-Driven Rules

Business events should be emitted for major state changes.

Examples:

- UserRegistered
- OrganizationVerified
- CampaignPublished
- ContributionVerified
- PaymentSucceeded
- VolunteerVerified
- TimeCreditsAwarded
- RewardRedeemed

Events trigger:

- Notifications
 - Search indexing
 - Analytics
 - Audit enrichment
-

16. Audit Rules

Every important action is audited.

Audit records include:

- Actor
- Target
- Action
- Before state
- After state
- Timestamp
- Request ID
- IP address
- User agent

Audit logs are immutable.

17. Coding Rules

Code must be:

- Strongly typed
- Modular
- Tested
- Documented
- Maintainable

Avoid duplication.

Prefer composition over inheritance.

Use dependency injection.

Keep business logic out of controllers.

18. Testing Rules

Every feature requires:

- Unit tests
- Integration tests
- API tests
- Frontend tests where applicable

Bug fixes require regression tests.

19. AI Development Workflow

For every development session:

1. Read this document.
2. Read the relevant Development Package.
3. Review existing code.
4. Produce an implementation plan.
5. Implement incrementally.
6. Run tests.
7. Update documentation.
8. Perform self-review.
9. Ensure all acceptance criteria are met before marking the task complete.

Never implement functionality outside the current Development Package.

20. Success Criteria

Every completed feature must:

- Compile successfully.
- Pass linting.
- Pass type checking.
- Pass automated tests.
- Follow the architecture.
- Preserve module boundaries.
- Update OpenAPI documentation.
- Update developer documentation.
- Be production-ready.

The primary objective is not to generate code quickly.

The primary objective is to build a secure, maintainable and scalable platform that can evolve over many years without major architectural refactoring.